

# 多租户

## 多租户

此页面概述了集群多租户的可用配置选项和最佳实践。

共享集群可以节省成本并简化管理。然而，共享集群也带来了诸如安全性、公平性和管理**嘈杂邻居**等挑战。

集群可以通过多种方式共享。在某些情况下，不同的应用可能会在同一个集群中运行。在其他情况下，同一应用的多个实例可能在同一个集群中运行，每个实例对应一个最终用户。所有这些类型的共享经常使用一个总括术语 **多租户 (Multi-Tenancy)** 来表述。

虽然 Kubernetes 没有最终用户或租户的一阶概念，它还是提供了几个特性来帮助管理不同的租户需求。下面将对此进行讨论。

## 用例 {#use-cases}

确定如何共享集群的第一步是理解用例，以便你可以评估可用的模式和工具。一般来说，Kubernetes 集群中的多租户分为两大类，但也可以有许多变体和混合。

### 多团队 {#multiple-teams}

多租户的一种常见形式是在组织内的多个团队之间共享一个集群，每个团队可以操作一个或多个工作负载。这些工作负载经常需要相互通信，并与位于相同或不同集群上的其他工作负载进行通信。

在这一场景中，团队成员通常可以通过类似 kubectl 等工具直接访问 Kubernetes 资源，或者通过 GitOps 控制器或其他类型的自动化发布工具间接访问 Kubernetes 资源。不同团队的成员之间通常存在某种程度的信任，但 RBAC、配额和网络策略等 Kubernetes 策略对于安全、公平地共享集群至关重要。

### 多客户 {#multiple-customers}

多租户的另一种主要形式通常涉及为客户运行多个工作负载实例的软件即服务 (SaaS) 供应商。这种业务模型与其部署风格之间的相关非常密切，以至于许多人称之为“SaaS 租户”。但是，更好的术语可能是“多客户租户 (Multi-Customer Tenancy)”，因为 SaaS 供应商也可以使用其他部署模型，并且这种部署模型也可以在 SaaS 之外使用。

在这种情况下，客户无权访问集群；从他们的角度来看，Kubernetes 是不可见的，仅由供应商用于管理工作负载。成本优化通常是一个关键问题，Kubernetes 策略用于确保工作负载彼此高度隔离。

## 术语 {#terminology}

### 租户 {#tenants}

在讨论 Kubernetes 中的多租户时，“租户”没有单一的定义。相反，租户的定义将根据讨论的是多团队还是多客户租户而有所不同。

在多团队使用中，租户通常是一个团队，每个团队通常部署少量工作负载，这些工作负载会随着服务的复杂性而发生规模伸缩。然而，“团队”的定义本身可能是模糊的，因为团队可能被组织成更高级别的部门或细分为更小的团队。

相反，如果每个团队为每个新客户部署专用的工作负载，那么他们使用的是多客户租户模型。在这种情况下，“租户”只是共享单个工作负载的一组用户。这种租户可能大到整个公司，也可能小到该公司的一个团队。

在许多情况下，同一组织可能在不同的上下文使用“租户”的两种定义。例如，一个平台团队可能向多个内部“客户”提供安全工具和数据库等共享服务，而 SaaS 供应商也可能让多个团队共享一个开发集群。最后，混合架构也是可能的，例如，某 SaaS 提供商为每个客户的敏感数据提供独立的工作负载，同时提供多租户共享的服务。

### 隔离 {#isolation}

使用 Kubernetes 设计和构建多租户解决方案有多种方法。每种方法都有自己的一组权衡，这些权衡会影响隔离级别、实现工作量、操作复杂性和服务成本。

Kubernetes 集群由运行 Kubernetes 软件的控制平面和由工作节点组成的数据平面组成，租户工作负载作为 Pod 在工作节点上执行。租户隔离可以根据组织要求应用于控制平面和数据平面。

所提供的隔离级别有时会使用一些术语来描述，例如“硬性（Hard）”多租户意味着强隔离，而“柔性（Soft）”多租户意味着较弱的隔离。特别是，“硬性”多租户通常用于描述租户彼此不信任的情况，并且大多是从安全和资源共享的角度（例如，防范数据泄露或 DoS 攻击等）。由于数据平面通常具有更大的攻击面，“硬性”多租户通常需要额外注意隔离数据平面，尽管控制平面隔离也很关键。

但是，“硬性”和“柔性”这两个术语常常令人困惑，因为没有一种定义能够适用于所有用户。相反，依据“硬度（Hardness）”或“柔度（Softness）”所定义的广泛谱系则更容易理解，根据你的需求，可以使用许多不同的技术在集群中维护不同类型的隔离。

在更极端的情况下，彻底放弃所有集群级别的共享并为每个租户分配其专用集群可能更容易或有必要，如果认为虚拟机所提供的安全边界还不够，甚至可以在专用硬件上运行。对于托管的 Kubernetes 集群而言，这种方案可能更容易，其中创建和操作集群的开销至少在一定程度上由云提供商承担。必须根据管理多个集群的成本和复杂性来评估更强的租户隔离的好处。Multi-Cluster SIG 负责解决这些类型的用例。

本页的其余部分重点介绍用于共享 Kubernetes 集群的隔离技术。但是，即使你正在考虑使用专用集群，查看这些建议也可能很有价值，因为如果你的需求或功能发生变化，它可以让你在未来比较灵活地切换到共享集群。

## 控制面隔离 {#control-plane-isolation}

控制平面隔离确保不同租户无法访问或影响彼此的 Kubernetes API 资源。

### 命名空间 {#namespaces}

在 Kubernetes 中，提供了一种在单个集群中隔离 API 资源组的机制。这种隔离有两个关键维度：

1. 一个命名空间中的对象名称可以与其他命名空间中的名称重叠，类似于文件夹中的文件。这允许租户命名他们的资源，而无需考虑其他租户在做什么。
2. 许多 Kubernetes 安全策略的作用域是命名空间。例如，RBAC Role 和 NetworkPolicy 是命名空间作用域的资源。使用 RBAC，可以将用户账号和服务账号限制在一个命名空间中。

在多租户环境中，命名空间有助于将租户的工作负载划分到各不相同的逻辑管理单元中。事实上，一种常见的做法是将每个工作负载隔离在自己的命名空间中，即使多个工作负载由同一个租户操作。这可确保每个工作负载都有自己的身份，并且可以使用适当的安全策略进行配置。

命名空间隔离模型需要配置其他几个 Kubernetes 资源、网络插件，并遵守安全最佳实践以正确隔离租户工作负载。这些考虑将在下面讨论。

### 访问控制 {#access-controls}

控制平面最重要的隔离类型是授权。如果各个团队或其工作负载可以访问或修改彼此的 API 资源，他们可以更改或禁用所有其他类型的策略，从而取消这些策略可能提供的任何保护。因此，确保每个租户只对他们需要的命名空间有适当的访问权，而不是更多，这一点至关重要。这被称为“最小特权原则（Principle of Least Privileges）”。

基于角色的访问控制 (RBAC) 通常用于在 Kubernetes 控制平面中对用户和工作负载（服务账号）强制执行鉴权。角色和角色绑定是两种 Kubernetes 对象，用来在命名空间级别对应用实施访问控制；对集群级别的对象访问鉴权也有类似的对象，不过这些对象对于多租户集群不太有用。

在多团队环境中，必须使用 RBAC 来限制租户只能访问合适的命名空间，并确保集群范围的资源只能由集群管理员等特权用户访问或修改。

如果一个策略最终授予用户的权限比他们所需要的还多，这可能是一个信号，表明包含受影响资源的命名空间应该被重构为更细粒度的命名空间。命名空间管理工具可以通过将通用 RBAC 策略应用于不同的命名空间来简化这些细粒度命名空间的管理，同时在必要时仍允许细粒度策略。

## 配额 {#quotas}

Kubernetes 工作负载消耗节点资源，例如 CPU 和内存。在多租户环境中，你可以使用资源配额来管理租户工作负载的资源使用情况。对于多团队场景，各个租户可以访问 Kubernetes API，你可以使用资源配额来限制租户可以创建的 API 资源的数量（例如：Pod 的数量或 ConfigMap 的数量）。对对象计数的限制确保了公平性，并有助于避免**嘈杂邻居**问题影响共享控制平面的其他租户。

资源配额是命名空间作用域的对象。通过将租户映射到命名空间，集群管理员可以使用配额来确保租户不能垄断集群的资源或压垮控制平面。命名空间管理工具简化了配额的管理。此外，虽然 Kubernetes 配额仅针对单个命名空间，但一些命名空间管理工具允许多个命名空间组共享配额，与内置配额相比，降低了管理员的工作量，同时为其提供了更大的灵活性。

配额可防止单个租户所消耗的资源超过其被分配的份额，从而最大限度地减少**嘈杂邻居**问题，即一个租户对其他租户工作负载的性能产生负面影响。

当你对命名空间应用配额时，Kubernetes 要求你还为每个容器指定资源请求和限制。限制是容器可以消耗的资源量的上限。根据资源类型，尝试使用超出配置限制的资源的容器将被限制或终止。当资源请求设置为低于限制时，每个容器所请求的数量都可以得到保证，但可能仍然存在跨工作负载的一些潜在影响。

配额不能针对所共享的所有资源（例如网络流量）提供保护。节点隔离（如下所述）可能是解决此问题的更好方法。

## 数据平面隔离 {#data-plane-isolation}

数据平面隔离确保不同租户的 Pod 和工作负载之间被充分隔离。

## 网络隔离 {#network-isolation}

默认情况下，Kubernetes 集群中的所有 Pod 都可以相互通信，并且所有网络流量都是未加密的。这可能导致安全漏洞，导致流量被意外或恶意发送到非预期目的地，或被受感染节点上的工作负载拦截。

Pod 之间的通信可以使用网络策略来控制，它使用命名空间标签或 IP 地址范围来限制 Pod 之间的通信。在需要租户之间严格网络隔离的多租户环境中，建议从拒绝 Pod 之间通信的默认策略入手，然后添加一条允许所有 Pod 查询 DNS 服务器以进行名称解析的规则。有了这样的默认策略之后，你就可以开始添加允许在命名空间内进行通信的更多规则。另外建议不要在网络策略定义中对 namespaceSelector 字段使用空标签选择算符“{}”，以防需要允许在命名空间之间传输流量。该方案可根据需要进一步细化。请注意，这仅适用于单个控制平面内的 Pod；属于不同虚拟控制平面的 Pod 不能通过 Kubernetes 网络相互通信。

命名空间管理工具可以简化默认或通用网络策略的创建。此外，其中一些工具允许你在整个集群中强制实施一组一致的命名空间标签，确保它们是你策略的可信基础。

网络策略需要一个支持网络策略实现的 CNI 插件。否则，NetworkPolicy 资源将被忽略。

服务网格可以提供更高级的网络隔离，除了命名空间之外，它还提供基于工作负载身份的 OSI 第 7 层策略。这些更高层次的策略可以更轻松地管理基于命名空间的多租户，尤其是存在多个命名空间专用于某一个租户时。服务网格还经常使用双向 TLS 提供加密能力，即使在存在受损节点的情况下也能保护你的数据，并且可以跨专用或虚拟集群工作。但是，它们的管理可能要复杂得多，并且可能并不适合所有用户。

## 存储隔离 {#storage-isolation}

Kubernetes 提供了若干类型的卷，可以用作工作负载的持久存储。为了安全和数据隔离，建议使用动态卷制备，并且应避免使用节点资源的卷类型。

存储类（StorageClass）允许你根据服务质量级别、备份策略或由集群管理员确定的自定义策略描述集群提供的自定义存储“类”。

Pod 可以使用持久卷申领（PersistentVolumeClaim）请求存储。

PersistentVolumeClaim 是一种命名空间作用域的资源，它可以隔离存储系统的不同部分，并将隔离出来的存储提供给共享 Kubernetes 集群中的租户专用。但是，重要的是要注意 PersistentVolume 是集群作用域的资源，并且其生命周期独立于工作负载和命名空间的生命周期。



例如，你可以为每个租户配置一个单独的 StorageClass，并使用它来加强隔离。如果一个 StorageClass 是共享的，你应该设置一个回收策略 以确保 PersistentVolume 不能在不同的命名空间中重复使用。

## 沙箱容器 {#sandboxing-containers}

Kubernetes Pod 由在工作节点上执行的一个或多个容器组成。容器利用操作系统级别的虚拟化，因此提供的隔离边界比使用基于硬件虚拟化的虚拟机弱一些。

在共享环境中，攻击者可以利用应用和系统层中未修补的漏洞实现容器逃逸和远程代码执行，从而允许访问主机资源。在某些应用中，例如内容管理系统（CMS），客户可能被授权上传和执行非受信的脚本或代码。无论哪种情况，都需要使用强隔离进一步隔离和保护工作负载的机制。

沙箱提供了一种在共享集群中隔离运行中的工作负载的方法。它通常涉及在单独的执行环境（例如虚拟机或用户空间内核）中运行每个 Pod。当你运行不受信任的代码时（假定工作负载是恶意的），通常建议使用沙箱，这种隔离是必要的，部分原因是由于容器是在共享内核上运行的进程。它们从底层主机挂载像 /sys 和 /proc 这样的文件系统，这使得它们不如在具有自己内核的虚拟机上运行的应用安全。虽然 seccomp、AppArmor 和 SELinux 等控件可用于加强容器的安全性，但很难将一套通用规则应用于在共享集群中运行的所有工作负载。在沙箱环境中运行工作负载有助于将主机隔离开来，不受容器逃逸影响，在容器逃逸场景中，攻击者会利用漏洞来访问主机系统以及在该主机上运行的所有进程/文件。

虚拟机和用户空间内核是两种流行的沙箱方法。

## 节点隔离 {#node-isolation}

节点隔离是另一种可用于将租户工作负载相互隔离的技术。通过节点隔离，一组节点专用于运行来自特定租户的 Pod，并且禁止混合不同租户 Pod 集合。这种配置减少了嘈杂的租户问题，因为在一个节点上运行的所有 Pod 都将属于一个租户。节点隔离的信息泄露风险略低，因为成功实现容器逃逸的攻击者也只能访问挂载在该节点上的容器和卷。

尽管来自不同租户的工作负载在不同的节点上运行，仍然很重要的是要注意 kubelet 和（除非使用虚拟控制平面）API 服务仍然是共享服务。熟练的攻击者可以使用分配给 kubelet 或节点上运行的其他 Pod 的权限在集群内横向移动并获得对其他节点上运行的租户工作负载的访问权限。如果这是一个主要问题，请考虑实施补偿控制，例如使用 seccomp、AppArmor 或 SELinux，或者探索使用沙箱容器，或者为每个租户创建单独的集群。

从计费的角度来看，节点隔离比沙箱容器更容易理解，因为你可以按节点而不是按 Pod 收费。它的兼容性和性能问题也较少，而且可能比沙箱容器更容易实现。例如，可以为每个租户的节点配置污点，以便只有具有相应容忍度的 Pod 才能在其上运行。然后可以使用变更性质的 Webhook 自动向部署到租户命名空间中的 Pod 添加容忍度和节点亲和性，以便它们在为该租户指定的一组特定节点上运行。

节点隔离可以使用将 Pod 指派给节点来实现。

## 额外的注意事项 {#additional-considerations}

本节讨论与多租户相关的其他 Kubernetes 结构和模式。

### API 优先级和公平性 {#api-priority-and-fairness}

API 优先级和公平性是 Kubernetes 的一个特性，允许你为集群中运行的某些 Pod 赋予优先级。当应用调用 Kubernetes API 时，API 服务器会评估分配给 Pod 的优先级。来自具有较高优先级的 Pod 的调用会在具有较低优先级的 Pod 的调用之前完成。当争用很激烈时，较低优先级的调用可以排队，直到服务器不那么忙，或者你可以拒绝请求。

使用 API 优先级和公平性在 SaaS 环境中并不常见，除非你允许客户运行与 Kubernetes API 接口的应用，例如控制器。

### 服务质量 (QoS) {#qos}

当你运行 SaaS 应用时，你可能希望能够为不同的租户提供不同的服务质量 (QoS) 层级。例如，你可能拥有具有性能保证和功能较差的免费增值服务，以及具有一定性能保证的收费服务层。幸运的是，有几个 Kubernetes 结构可以帮助你在共享集群中完成此任务，包括网络 QoS、存储类以及 Pod 优先级和抢占。这些都是为了给租户提供他们所支付的服务质量。让我们从网络 QoS 开始。

通常，节点上的所有 Pod 共享一个网络接口。如果没有网络 QoS，一些 Pod 可能会以牺牲其他 Pod 为代价不公平地消耗可用带宽。Kubernetes 带宽插件为网络创建扩展资源，以允许你使用 Kubernetes 的 resources 结构，即 requests 和 limits 设置。通过使用 Linux tc 队列将速率限制应用于 Pod。请注意，根据支持流量整形文档，该插件被认为是实验性的，在生产环境中使用之前应该进行彻底的测试。

对于存储 QoS，你可能希望创建具有不同性能特征的不同存储类或配置文件。每个存储配置文件可以与不同的服务层相关联，该服务层针对 IO、冗余或吞吐量等不同的工作负载进行优化。可能需要额外的逻辑来允许租户将适当的存储配置文件与其工作负载相关联。

最后，还有 Pod 优先级和抢占，你可以在其中为 Pod 分配优先级值。在调度 Pod 时，当没有足够的资源来调度分配了较高优先级的 Pod 时，调度程序将尝试驱逐具有较低优先级的 Pod。如果你有一个用例，其中租户在共享集群中具有不同的服务层，例如免费和付费，你可能希望使用此功能为某些层级提供更高的优先级。

## DNS

Kubernetes 集群包括一个域名系统（DNS）服务，可为所有服务和 Pod 提供从名称到 IP 地址的转换。默认情况下，Kubernetes DNS 服务允许在集群中的所有命名空间中进行查找。

在多租户环境中，租户可以访问 Pod 和其他 Kubernetes 资源，或者在需要更强隔离的情况下，可能需要阻止 Pod 在其他名称空间中查找服务。你可以通过为 DNS 服务配置安全规则来限制跨命名空间的 DNS 查找。例如，CoreDNS（Kubernetes 的默认 DNS 服务）可以利用 Kubernetes 元数据来限制对命名空间内的 Pod 和服务的查询。有关更多信息，请阅读 CoreDNS 文档中配置此功能的示例。

当使用各租户独立虚拟控制面模型时，必须为每个租户配置 DNS 服务或必须使用多租户 DNS 服务。参见一个 CoreDNS 的定制版本支持多租户的示例。

## Operators

Operator 模式是管理应用的 Kubernetes 控制器。Operator 可以简化应用的多个实例的管理，例如数据库服务，这使它们成为多消费者 (SaaS) 多租户用例中的通用构建块。

在多租户环境中使用 Operators 应遵循一套更严格的准则。具体而言，Operator 应：

- 支持在不同的租户命名空间内创建资源，而不仅仅是在部署 Operator 的命名空间内。
- 确保 Pod 配置了资源请求和限制，以确保调度和公平。
- 支持节点隔离、沙箱容器等数据平面隔离技术的 Pod 配置。

## 实现 {#implementations}

为多租户共享 Kubernetes 集群有两种主要方法：使用命名空间（即每个租户独立的命名空间）或虚拟化控制平面（即每个租户独立的虚拟控制平面）。

在这两种情况下，还建议对数据平面隔离和其他考虑事项，如 API 优先级和公平性，进行管理。



Kubernetes 很好地支持命名空间隔离，其资源开销可以忽略不计，并提供了允许租户适当交互的机制，例如允许服务之间的通信。但是，它可能很难配置，而且不适用于非命名空间作用域的 Kubernetes 资源，例如自定义资源定义、存储类和 Webhook 等。

控制平面虚拟化允许以更高的资源使用率和更困难的跨租户共享为代价隔离非命名空间作用域的资源。当命名空间隔离不足但不希望使用专用集群时，这是一个不错的选择，因为维护专用集群的成本很高（尤其是本地集群），或者由于专用集群的额外开销较高且缺乏资源共享。但是，即使在虚拟化控制平面中，你也可能会看到使用命名空间的好处。

以下各节将更详细地讨论这两个选项：

### 每个租户独立的命名空间 {#namespace-per-tenant}

如前所述，你应该考虑将每个工作负载隔离在其自己的命名空间中，即使你使用的是专用集群或虚拟化控制平面。这可确保每个工作负载只能访问其自己的资源，例如 ConfigMap 和 Secret，并允许你为每个工作负载定制专用的安全策略。此外，最佳实践是为整个集群中的每个命名空间名称提供唯一的名称（即，即使它们位于单独的集群中），因为这使你将来可以灵活地在专用集群和共享集群之间切换，或者使用多集群工具，例如服务网格。

相反，在租户级别分配命名空间也有优势，而不仅仅是工作负载级别，因为通常有一些策略适用于单个租户拥有的所有工作负载。然而，这种方案也有自己的问题。首先，这使得为各个工作负载定制策略变得困难或不可能，其次，确定应该赋予命名空间的单一级别的“租户”可能很困难。例如，一个组织可能有部门、团队和子团队 - 哪些应该分配一个命名空间？

一种可能的方法是将多个命名空间组织成层次结构，并在它们之间共享某些策略和资源。这可以包括管理命名空间标签、命名空间生命周期、委托访问权限，以及在相关命名空间之间共享资源配额。这些功能在多团队和多客户场景中都很有用。

### 每个租户独立的虚拟控制面 {#virtual-control-plane-per-tenant}

控制面隔离的另一种形式是使用 Kubernetes 扩展为每个租户提供一个虚拟控制面，以实现集群范围内 API 资源的分段。数据平面隔离技术可以与此模型一起使用，以安全地跨多个租户管理工作节点。

基于虚拟控制面的多租户模型通过为每个租户提供专用控制面组件来扩展基于命名空间的多租户，从而完全控制集群范围的资源和附加服务。工作节点在所有租户之间共享，并由租户通常无法访问的 Kubernetes 集群管理。该集群通常被称为 **超集群 (Super-**

**Cluster**) (或有时称为 **host-cluster**)。由于租户的控制面不直接与底层计算资源相关, 因此它被称为**虚拟控制平面**。

虚拟控制面通常由 Kubernetes API 服务器、控制器管理器和 etcd 数据存储组成。它通过元数据同步控制器与超集群交互, 该控制器跨租户控制面和超集群控制面对变化进行协调。

通过使用每个租户单独的专用控制面, 可以解决由于所有租户共享一个 API 服务器而导致的大部分隔离问题。例如, 控制平面中的嘈杂邻居、策略错误配置导致的不可控爆炸半径以及如 Webhook 和 CRD 等集群范围对象之间的冲突。因此, 虚拟控制平面模型特别适用于每个租户都需要访问 Kubernetes API 服务器并期望具有完整集群可管理性的情况。

改进的隔离是以每个租户运行和维护一个单独的虚拟控制平面为代价的。此外, 租户层面的控制面不能解决数据面的隔离问题, 例如节点级的嘈杂邻居或安全威胁。这些仍然必须单独解决。